Correctness: **Author: Yue (Rosy) Yu**

- data structures → invariant (the invariant holds before an operation is executed, the operation has the desired result and the invariant still holds after the operation is executed)
- greedy → exchange argument (see 9-Greedy)
- divide and conquer → induction (recursion)

1 - Time

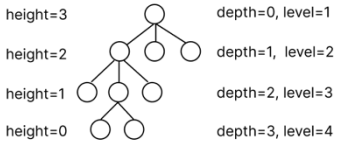
constant	$T(n) = \Theta(1)$
logarithmic	$T(n) = \Theta(\log n)$
linear	$T(n) = \Theta(n)$
quasi-linear	$T(n) = \Theta(n \log n)$
quadratic	$T(n) = \Theta(n^2)$
cubic	$T(n) = \Theta(n^3)$
exponential	$T(n) = \Theta(c^n)$

2 – Lists initialize -> O(1)

Data Structure	Operation	Time
Array	`get(i)`, `set(i, e)`	O(1)
	`insert(0, e)`, `remove(0)` from the start	O(n)
	`insert(i, e)`, `remove(i)` in the middle	O(n)
	`append(e)`, `pop()` from the end	O(1)
	`replace(i, e)`, `size()`, `isEmpty()`	O(1)
Singly Linked	`get(i)`	O(n)
/ Doubly Linked	`addFirst(e)`, `removeFirst()` from start	O(1)
	`add(i, e)`, `remove(i)` in middle	O(n)
	`addLast(e)`, `removeLast()` from end	O(n) / O(1)
	`replace(i, e)`	O(n)
	`size()`	O(n)
Stack	push(e), pop()	O(1)
Queue	enqueue(e), dequeue()	O(1)

3 – Trees Preorder, Inorder, Postorder → O(n)

Method	Time
`size()`, `isEmpty()`	O(1)
`root()`, `parent(p`	O(1)
`children(p)`	O(c)
`numChildren(p)`	O(1)
`isInternal(p)`, `isExternal(p)`, `isRoot(p)`	O(1)



Root: height=3, depth=0, level=1

4 - Binary Search Tree should consider h's best & worst case

Binary Search Tree (BST)

1. **Properties**:
   - For any node v, keys in the left subtree are less than v, and keys in the right subtree are greater than v.
   - In-order traversal of a BST results in keys in increasing order.
2. **Operations**:
   - **Search**: `search(k, v)` O(h) (h = height)
   - **Insertion**: `put(k, o)` O(h)
     - If the key exists, replace the value.
     - Otherwise, insert at the reached external node.
   - **Deletion**: `remove(k)` O(h)
     - **Case 1**: Node has one external child — remove and promote the other child.
     - **Case 2**: Node has two internal children — find the in-order successor, copy its value, and delete it.
3. **Complexity**:
   - **Space**: O(n)
   - **Best Case Height**: O(log n)
   - **Worst Case Height**: O(n)

Range Queries
(to find all keys k stored in T such that k1 ⩽ k ⩽ k2.):

```
def range(v, k1, k2)
  if v is external then
    return null
  if key(v) > k2 then
    range(v.left, k1, k2)
  else if key(v) < k1 then
    range(v.right, k1, k2)
  else
    range(v.left, k1, k2)
    output.append(v)
    range(v.right, k1, k2)
```

time complexity: O(|output| + h)

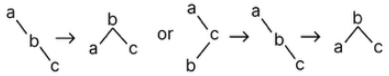
AVL all operations in O(log n).

Index-based searching in AVL: select(i) returns the ith smallest key (assume no duplicate keys)

```
def select(x, i):
  input A node x of an AVL tree
        integer i : 1 ≤ i ≤ size(x) position in T_x to select
  output ith element in T_x (i=1 is first key)

  if i ≤ size(x.left) then
    return select(x.left, i)
  else if i = size(x.left) + 1 then
    return x.key
  else
    return select(x.right, i − size(x.left) − 1)
```

trinode restructuring



5 - Priority Queues

Method	Unsorted List	Sorted List	Heap
size, isEmpty	$O(1)$	$O(1)$	$O(1)$
insert	$O(1)$	$O(n)$	$O(\log n)$
min	$O(n)$	$O(1)$	$O(1)$
removeMin	$O(n)$	$O(1)$	$O(\log n)$
remove	$O(1)$	$O(1)$	$O(\log n)$
replaceKey	$O(1)$	$O(n)$	$O(\log n)$
replaceValue	$O(1)$	$O(1)$	$O(1)$

- Selection sort: for each i in the list, find the min value s after i, swap s and i.

- Insertion sort: for each i in the list, shifts a chunk of consecutive larger numbers before i to i, then insert i at the start of the consecutive larger numbers.

Min-Heap Operations:

Operation	Description	Time
Build Heap	Builds a heap from an unsorted array	O(n)
`insert`	Insert at the last position and perform **upheap**	O(logn)
`remove_min`	Replace the root with the last node, remove the last node, and perform **downheap**	O(logn)
Access root `min`	Returns the min/max element without removal	O(1)
Decrease/Increase Key	Adjusts the value of a specific key	O(logn)
Find Next Last Node After Deletion	Start from the old last node, go up to find a right child or root, go to sibling, then down to leaf	O(logn)

Heap to array: root is at 0, last node is at n-1. For the node at index i, its left child is at index 2i+1, its right child is at index 2i+2, its parent is at index ⌊(i-1)/2⌋.

Use heapify to build heap in O(n): build heap from bottom to up, downheap the internal node when adding it.
Time complexity proof: every internal node goes downheap in a unique path, there are n-1 edges in a complete binary tree, so O(n).

6 - Hashing

- Unordered key list: O(1) for insert, O(n) for get and delete.
- Order key list: waste of space.
- Goal: want hashmap of all O(1) operations.

- Maps n keys to the range [0, N-1], the expected number of collisions is at most the load factor a = n/N.
- Use polynomial evaluation function as the hash function when the order of permutation matters.

Separate chaining(an array of linked lists): load factor is the average number of keys per bucket, If the hash function distributes keys uniformly, the expected time per operation is O(1 + a), but normally worst case running time is O(n).

linear probing(place key to next empty slot, circularly):
If the load factor is less than 1, put/get would run in O(1).
P(next slot is empty, success) = 1-a, P(next slot is filled, fail) = a, the expected number of trials until the first success is 1/p, therefore the average number of probes you must make before hitting an empty slot is 1/(1-a). So with a ⩽ 0.5, you expect about 2 probes per operation.
Deletion leaves a DEFUNCT marker, so that we don't break the chain when finding a key.

Cuckoo Hashing (two slots to put a key, evict the key to its another slot if filled): O(1) for get and remove, O(n) for put.

To detect an eviction cycle: use a counter to count how many times this key was evicted, has a cycle if evicted enough time; or keep a flag for each key, flag it when we evicted it, unflag all when successfully put.

7 – Graphs

▪ *n* vertices, *m* edges ▪ no parallel edges ▪ no self-loops	Edge List	Adjacency List	Adjacency Matrix
Space	$O(n + m)$	$O(n + m)$	$O(n^2)$
incidentEdges(**v**)	$O(m)$	O(deg(v))	$O(n)$
getEdge(**u, v**)	$O(m)$	O(min(deg(**u**), deg(v)))	O(1)
insertVertex(**x**)	O(1)	O(1)	$O(n^2)$
insertEdge(**u, v, x**)	O(1)	O(1)	O(1)
removeVertex(**v**)	$O(m)$	O(deg(v))	$O(n^2)$
removeEdge(**e**)	O(1)	O(1)	O(1)

if Adjacency List need traverse, `removeVertex`: O(n+m) `removeEdge`: O(m)

Graph Traversal

Algorithm	Complexity (Adjacency List)	Edges
DFS	O(n+m)	back edges
BFS	O(n+m)	cross edges

Applications	DFS	BFS
Spanning forest, connected components, paths, cycles	√	√
Shortest paths		√
Cut edges	√	

BFS can find bipartiteness and shortest path (if all weight = 1).
DFS can find cut edges (for (u,v), is a cut edge if low[v] < level[u], where low[v] is the lowest level that v can reach by going down a path and a back edge) and cycles.

Topological sort can return a list in the order of dependency and fills prerequisites for any scheduling tasks.
Every Directed Acyclic Graph can be topologically sorted, A topological order is a linear ordering of vertices so that for every edge (u,v), u appears before v.
Run DFS to check the cycle and record finished time of each vertex, build the sorted list on the fly in decreasing finish-time order, O(n+m) running time.

Strongly Connected Components (SCCs): In a directed graph G, a strongly connected component is a subset C where every vertex in C can reach every other vertex in C:

For any u,v ∈ C, there is a u-v path in G

No superset of C has the above property

We can then collapse each SCC into a single node, then get a compressed DAG. DFS-based algorithms of SCC have O(n+m) running time and can return all compressed nodes.

Author: Yue (Rosy) Yu

```
def SCC(G):
    DFS(G)
    F ← copy of G with reversed edges
    run DFS on F but process vertices in
      reverse order of DFS-visit call finish
    components ← []
    for tree T in DFS(F) forest:
       components.append(vertices in T)

    return components
```

8 - shortest path and MST

A subpath of a shortest path is itself a shortest path.

Dijkstra's Algorithm: Compute the shortest path from a start vertex s to every other vertex in a graph with nonnegative edge weights. Always pick the not-yet-settled vertex with the smallest accumulated distance from s. Use a heap to store (v, D[v]) for v in G, and update D[v] and decrease key when we relax edge. O(m log n) with a binary heap, or O(m + n log n) with a Fibonacci heap.

Shortest Path Problem with Edge Weights and Vertex Costs

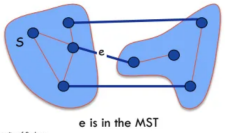
- Create a new graph G' by modifying each edge (u,v) in G.
- In G', set the weight of (u',v') to **edge weight + cost of the destination vertex.**
- **Run Dijkstra's Algorithm** on G'

Minimum Spanning Tree

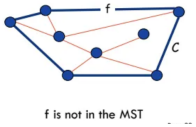
Simplifying assumption. All edge costs c_e are distinct.

Cut property. Let S be any subset of nodes, and let e be the min cost edge with exactly one endpoint in S. Then the MST contains e.

Cycle property. Let C be any cycle, and let f be the max cost edge belonging to C. Then the MST does not contain f.



e is in the MST



f is not in the MST

Cycle-cut intersection claim:
A cycle and a cutset intersect in an even number of edges.

Fibonacci heap: amortize O(1) insert; O(1) Find-min; O(1) Merge (union) two heaps; amortized O(1) Decrease-key; amortized O(log n) Extract-min (delete-min).

Prim's Algorithm(grow a tree): start with any random vertex v, always expand the lightest edge until all nodes are expanded. O(m log n) with a binary heap, or O(m + n log n) with a Fibonacci heap. Prove by cut property.

Tree-based implementation of union-find(with path compression):
initialize every vertex v to be the root of a singleton tree, each root contains a variable 'size' indicating the current size of this tree, initialized to be 1. If we want to add an edge (u, v), we do a = find(u), b = find(v), and check if a != b. (During a find(u), follow parent pointers up to the root, then set each visited node's parent directly to the root. find(u) return the root of the tree u is in) If a != b, we do union(a, b), which compares the size of a, b, and moves the tree with smaller size to the other tree. If a.size < b.size, append a to be the direct child of b, and update 'size' of b to be b+a.

Kruskal's Algorithm

- Steps:
  1. Sort all edges by weight.
  2. For each edge, check if it forms a cycle with the current MST using Union-Find.
  3. If no cycle is formed, add the edge to the MST.
  4. Stop once n−1 edges are added.
- Time Complexity:
  - Sorting edges: O(mlogm).
  - Using Union-Find to check cycles: O(n^2)
  - `make_sets(A)`: O(n) `find(u)`: O(1) `union(u, v)`: O(n)

Running time of tree-based implementation of union-find is O(m a(n)) for m finds/union operations. a(n) is the Inverse Ackerman function. For α(n) to be larger than 4, you need n to be much larger than the number of subatomic particles in the universe, therefore it is a constant in practice.

9 -Greedy Algorithms

Greedy algorithm: always pick the best choice. Heuristic: **benefit-to-weight ratio** (bi/wi)

Allocating classrooms: Sort all lectures by increasing start time. Maintain a min-heap (priority queue) of end-times, one node per active classroom. For each lecture in start-time order, look at the earliest finishing classroom (the top of the heap): If that classroom is free (its finish time ≤ current lecture's start time), assign the lecture there (pop and push the new finish time). Otherwise, allocate a new classroom (push its finish time). The heap size at the end is the total rooms needed. O(n log n)

Text compression(Huffman Coding): sort the frequency fi of all distinct symbols in text T in increasing order. Initialize a min priority queue Q containing every distinct symbol i as a singleton tree. In each iteration, pop two smallest subtrees fi and fj from Q, merge them f = fi + fj, put it back to Q. At the end, the last tree in Q is the desired encoding tree. O(n log n)

Exchange argument:

– we have our greedy solution G

– Assume for simplicity that all ratios are different bi/wi ≠ bk/wk

– Consider some optimal solution X = {x1, x2, …, xm} different than the greedy one

– There must be item in G but not full in X, and item in X but not in G

– There must be items i and k, s.t. xi<wi, xk>0, and bi/wi > bk/wk.

– We try to transform X into G by canceling these differences

– replace min{wi-xi, xk} amount of item k with item i in X.

– since bi/wi > bk/wk, the net gain of X is bi/wi - bk/wk, which is a positive number.

– Turning X into G would strictly increase the total benefit, thus no better solution than G.

10 - Divide and Conquer I

Quick sort:

1. Choose a random element from the list as the pivot. Partition the elements into 3 lists:
(i) less than, (ii) equal to and (iii) greater than the pivot
2. Recursively sort the less than and greater than lists
3. Join the sorted 3 lists together

Harmonic number:

$$H(n) = \sum_{i=1}^{n} \frac{1}{i} = O(\log n)$$

Recurrence	Solution
T(n) = 2 T(n/2) + O(n)	T(n) = O(n log n)
T(n) = 2 T(n/2) + O(log n)	T(n) = O(n)
T(n) = 2 T(n/2) + O(1)	T(n) = O(n)
T(n) = T(n/2) + O(n)	T(n) = O(n)
T(n) = T(n/2) + O(1)	T(n) = O(log n)
T(n) = T(n-1) + O(n)	T(n) = O(n^2)
T(n) = T(n-1) + O(1)	T(n) = O(n)

11 - Divide and Conquer II

Maxima-set algorithm

(Definition: A point is maximum in a set if all other points in the set have either a smaller x- or smaller y-coordinate.):
Recur each left and right MS from the base case.
1. Find the highest point p in the Right MS
2. Compare every point q in the Left MS to this point. If q.y > p.y, add q to the Merged MS
3. Add every point in the Right MS to the Merged MS

Geometric series:
Let r be positive real and k a positive integer:

$$1 + r + r^2 + \dots + r^k = (r^{k+1} - 1)/(r-1)$$

If r > 1:

$$1 + r + r^2 + \dots + r^k < r^{k+1} / (r-1)$$

If r < 1:

$$1 + r + r^2 + \dots + r^k < 1 / (1-r)$$

Change base Log rule:

$$3^{\log_2 n} = e^{\ln(3)\frac{\ln n}{\ln 2}} = n^{\ln(3)/\ln(2)} = n^{\log_2 3}$$

Unrolling:

$$\sum_{k=0}^{\log_b n} a^k f(n/b^k)$$

Master Theorem

Let f(n) and T(n) be defined as follows:

$$T(n) = \begin{cases} a\,T(n/b) + f(n) & \text{for } n \geq d \\ c & \text{for } n < d \end{cases}$$

Depending on a, b and f(n) the recurrence solves to:

1. if $f(n) = O(n^{\log_b a - \varepsilon})$ for $\varepsilon > 0$ then $T(n) = \Theta(n^{\log_b a})$,
2. if $f(n) = \Theta(n^{\log_b a} \log^k n)$ for $k \geq 0$ then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$,
3. if $f(n) = \Omega(n^{\log_b a + \varepsilon})$ and $a\,f(n/b) \leq \delta\,f(n)$ for $\varepsilon > 0$ and $\delta < 1$ then $T(n) = \Theta(f(n))$,

Note: If f(n) is given as big-O, you can only conclude T(n) as big-O (not Θ).

Find k^th smallest element in unsorted array A in O(n):

1. **Divide** find the median (floor(n/2) th element for simplicity) and split array on the halves, ≤ and > than the median
2. **Recur**
   if k ≤ n/2: find k^th element on smaller half
   if k > n/2: find (k- n/2)th element on larger half
3. **Conquer** return value of the recursive call

Tree traversal

(pre-order)

```
def pre_order(v)
    visit(v)
    for each child w of v
        pre_order(w)
```

(post-order)
Useful when you need information from all subtrees before handling the parent

```
def post_order(v)
    for each child w of v
        post_order(w)
    visit(v)
```

(inorder - for binary tree)

```
def in_order(v)
    if v.left ≠ null then
        in_order(v.left)
    visit(v)
    if v.right ≠ null then
        in_order(v.right)
```